How to test Motion2Action live

From a tutorial to evidence-linked knowledge and real actions.

Recommended access: a supervised test on the project author's prepared Mac. Ask the author at the event to open the app. No app login is required on that prepared machine; provider credentials are already configured locally and are not shared. There is **no publicly hosted demo**. The localhost links below work only on the machine running the services.

1. Inspect the learned workflow — no desktop actions

01 Open Learning Showcase

On the prepared Mac, visit `http://127.0.0.1:3000/learning`. Select **Bevel Modifier: Round a Cube (Blender 4.4)** in Saved workflows.

02 Check what was learned and why

Read the six semantic steps. Click a timestamp to seek in the original video. Expand **Expected result & source evidence** to inspect the cited frames. Expand **How this learning is saved**, then choose **Download knowledge JSON** to inspect the persisted package.

Expected: source-linked instructions, expected outcomes, timestamps and frame evidence. *Source checked* means model-checked support in the video, not successful execution. Reading, seeking and exporting do not send desktop input.

2. Run the agent — explicit opt-in, real Blender input

03 Approve a new scratch execution

Ask the author to confirm the Mac is unlocked and the desktop is idle. Review the displayed steps, then click **Run with Agent**. This action approves a fresh scratch Blender scene and real mouse/keyboard input. Do not start another assessment or use the desktop during execution.

04 Observe and inspect the new receipt

Watch Blender open and the agent choose actions from current screenshots. In the app, inspect the new run ID, latest screenshot and **Recorded activity**. A reused workflow creates a new execution; it does not inherit an older result. Runtime varies with provider latency; there is no fixed completion-time promise.

05 Check the result, not just the clicks

A successful supported run saves `replica.blend` through the GUI, then a separate Blender process reads it and compares it with source-derived targets. For this learned workflow, targets include a cube, vertex Bevel, width 0.24, four segments and smooth shading. **Only matching source-bound artifact checks justify verified status.**

Stop or failure: choose **Stop Agent** when available. Stop is cooperative; an in-flight request may finish first. A blocked, stopped or failed run is not a pass. Ask the author to inspect its receipt before retrying; do not launch overlapping runs.

Evidence available: a retained execution passed 11 artifact checks and its saved file passed a fresh read-only inspection. This does not guarantee that every new run succeeds. See the repository's evaluation page for the exact scope and sanitized receipt.

Alternative tests & independent setup

3. Explore the software-assessment workspace

Visit `http://127.0.0.1:3000`. Ask the author to open a retained assessment from **Recent runs**. Inspect the relevant sources, recorded evidence, interface exploration, knowledge graph and report. This is read-only review of existing evidence.

To start a new assessment, use only an authorized installed application or public website and a scoped objective. **Start assessment can launch real exploration**; do not run it while tutorial execution is active. Non-Blender findings remain observed/source-backed unless an independent check exists. This is decision support, not formal UAT or procurement approval.

4. Run a fresh copy on your own Mac

Repository: `https://github.com/Hippolyte-Hu/appdna`

Use the **main** branch. The repository is public; no GitHub invitation is required. A fresh clone has no private recordings, saved catalog or local database.

Prerequisites: macOS; Python 3.11 and uv; Node.js 22/npm; FFmpeg/FFprobe; Blender at its standard Applications path; Xcode Command Line Tools/Swift and macOS `sandbox-exec`. Grant Accessibility and Screen Recording permissions to the launching runtime. Use your own H Company, Gladia and Linkup keys for the corresponding live stages; `yt-dlp` is needed for YouTube ingestion, not local uploads.

```
git clone https://github.com/Hippolyte-Hu/appdna.git
cd appdna
git switch main
uv venv --python 3.11 backend/.venv
uv pip install --python backend/.venv/bin/python -r backend/requirements.txt
cp backend/.env.example backend/.env
# Fill backend/.env privately; never commit credentials.
backend/.venv/bin/python -m playwright install chromium
cd frontend && npm ci
```

Start each service in its own terminal, from the indicated repository subdirectory:

```
# Terminal A -- backend/
PYTHONPATH=. .venv/bin/uvicorn app.main:app --host 127.0.0.1 --port 8787
# Terminal B -- backend/
APPDNA_LEARNING_MODEL=holo4-27b APPDNA_ACTION_MODEL=holo4-27b \
 PYTHONPATH=. .venv/bin/uvicorn app.tutorial_app:app --host 127.0.0.1 --port 8788
# Terminal C -- frontend/
npm run dev -- --hostname 127.0.0.1 --port 3000
```

On a new installation, open `http://127.0.0.1:8788` to learn an authorized tutorial first. Supply one YouTube video or MP4/MOV/WebM/MKV upload with spoken audio, at most 600 seconds; local uploads are limited to 100 MiB. Accepted knowledge then appears in Learning Showcase. Learning uses provider credits and can block on ambiguous evidence or provider failures.

5. Evidence, limits and troubleshooting

Read the source: README / full setup, workflow architecture, and evaluation / verification evidence. Regression checks: 208 backend and 13 frontend tests passed; these are not a live reliability benchmark.

No catalog: expected on a fresh clone; learn a source first. **Service unavailable:** check all three terminals and ports. **Run blocked:** check desktop lock, permissions, Blender, active runs and provider errors; keep safety gates enabled. Tutorial execution is Blender-only, with the bounded `blender_bevel_v1` verifier. Selected text, audio and images are sent to configured cloud providers. Bind locally; do not expose desktop-control services publicly.